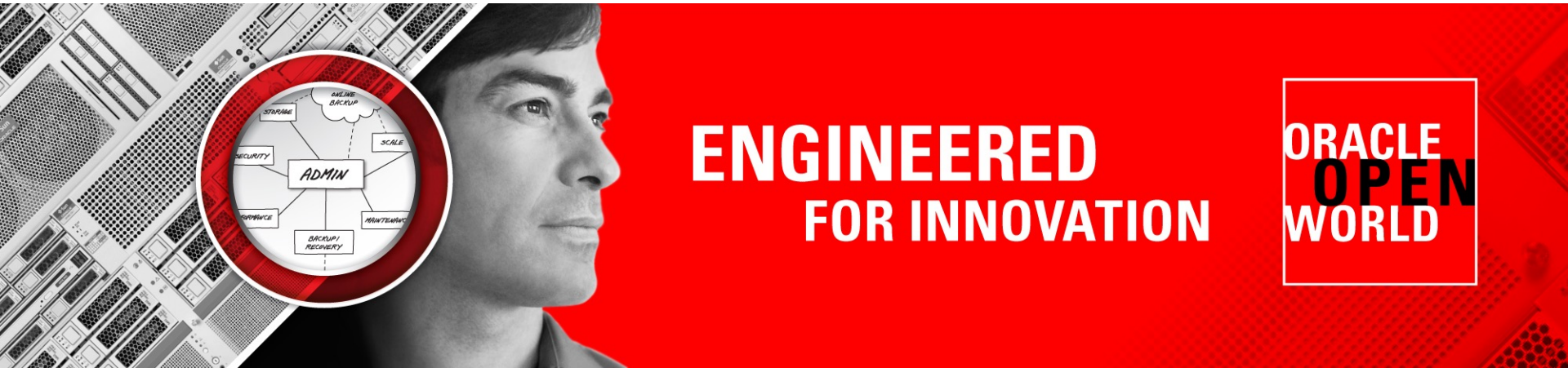




Partie 2 PL/SQL

Présenté par: Mme Olfa DRIDI



Plan de la 1^{ère} partie du cours

1. PLSQL
2. Bloc PLSQL
3. Déclaration des variables
4. Traitements conditionnels
5. Traitements itératifs
6. Curseurs
7. Procédures et fonctions
8. Gestion des erreurs
9. Déclencheurs



Bloc PL/SQL et variables

SQL mais

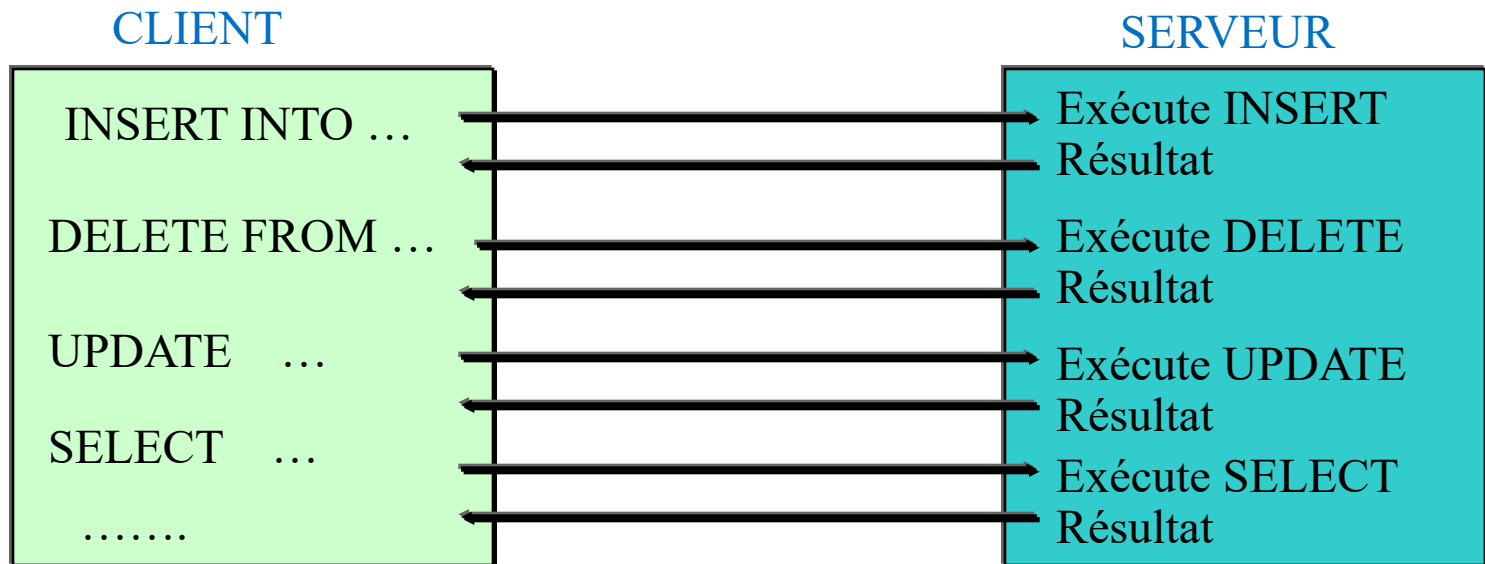
- SQL est déclaratif
 - Requêtes simples
 - Mais les applications complexes exigent plus,
 - Pour la facilité et l'efficacité de développement :
 - lier plusieurs requêtes entre elles,
 - créer des bibliothèques de procédures cataloguées réutilisables
 - Pour l'efficacité de l'application :
 - réduire les échanges client et serveur (un programme PL/SQL est exécuté sur le serveur)
- ⇒ Besoin d'étendre SQL :
PL/SQL est une extension procédurale

Langage PL/SQL

- PL/SQL est un langage de programmation procédural et structuré.
- Ce langage est une extension de SQL car il permet de faire cohabiter des structures de contrôle (*si*, *pour* et *tant que*) avec des instructions SQL (principalement SELECT, INSERT, UPDATE et DELETE).

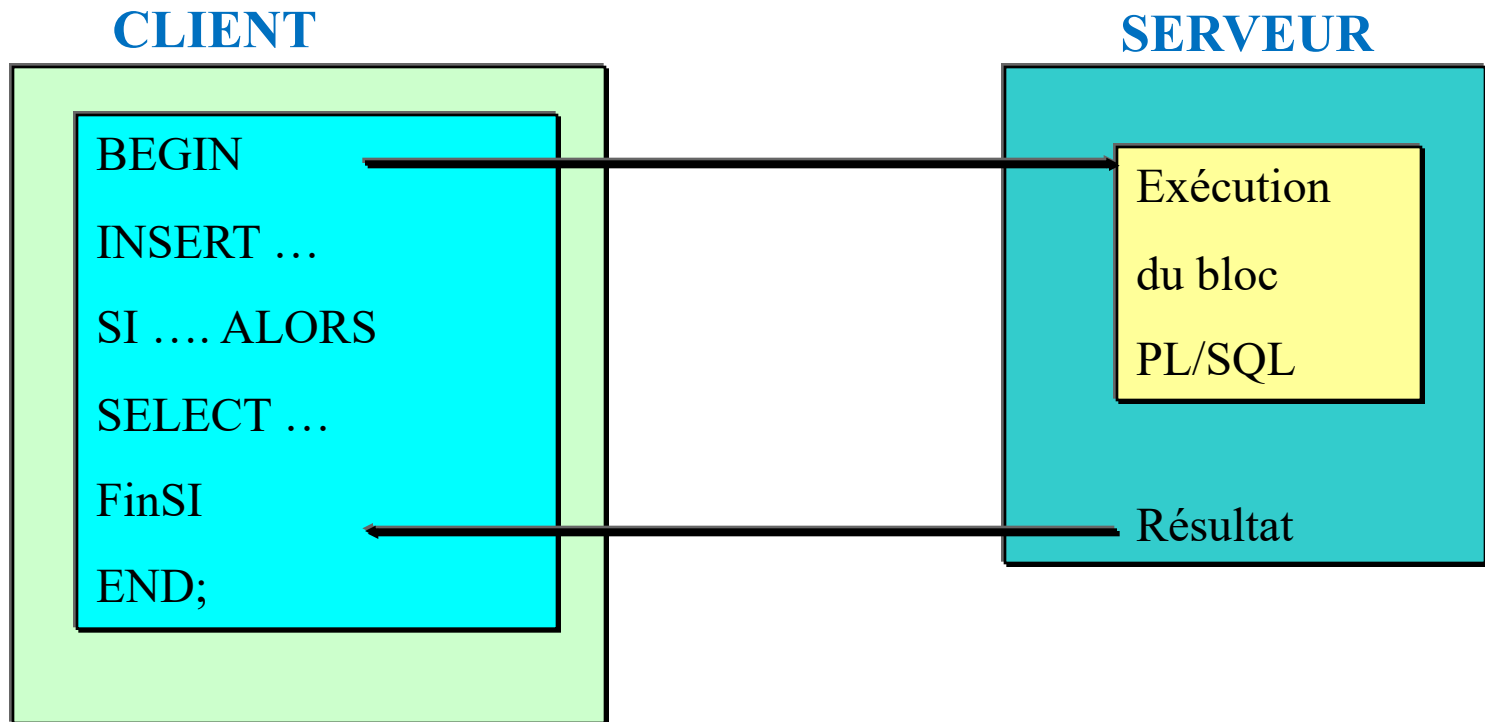
Requêtes SQL

- Chaque requête 'client' est transmise au serveur de données pour être exécutée avec retour de résultats



Bloc PL/SQL

- Le bloc de requêtes est envoyé sur le serveur. Celui-ci exécute le bloc et renvoie 1 résultat final.



Avantages de PL/SQL

- La **modularité** (un bloc d'instruction peut être composé d'un autre, etc.) : un bloc peut être nommé pour devenir une procédure ou une fonction, donc réutilisable. Une procédure, ou fonction, peut être incluse dans un paquetage (*package*) pour mieux contrôler et réutiliser ces composants logiciels.
- La **portabilité** : un programme PL/SQL est indépendant du système d'exploitation qui héberge le serveur Oracle. En changeant de système, les applicatifs n'ont pas à être modifiés.
- L'**intégration** avec les données des tables : on retrouvera avec PL/SQL tous les types de données et instructions disponibles sous SQL, et des mécanismes pour parcourir des résultats de requêtes, pour traiter des erreurs (exceptions), pour manipuler des données complexes (paquetages) et pour programmer des transactions (**COMMIT**, **ROLLBACK**, **SAVEPOINT**).

Structure d'un bloc PL/SQL

- Section DECLARE : déclaration de
 - Variables locales simples
 - Variables tableaux
 - curseurs
- Section BEGIN
 - Section des ordres exécutables
 - Ordres SQL
 - Ordres PL
- Section EXCEPTION
 - Réception en cas d'erreur
 - Exceptions SQL ou utilisateur

```
DECLARE
    --déclarations
BEGIN
    --exécutions
EXCEPTION
    --erreurs
END;
/
```

Types de bloc

Anonymous

```
[DECLARE]

BEGIN
    --statements

[EXCEPTION]

END;
```

Procedure

```
PROCEDURE name
IS

BEGIN
    --statements

[EXCEPTION]

END;
```

Function

```
FUNCTION name
RETURN datatype
IS
BEGIN
    --statements
    RETURN value;
[EXCEPTION]

END;
```

Variables

- Toute variable PL/SQL :
 - Obligatoirement définie dans **DECLARE** avant utilisation
 - Pas de déclarations multiples
- Types de variables PL/SQL :
 - Types Scalaires (Oracle) :
 - NUMBER, VARCHAR2, DATE, BOOLEAN, ...
 - Composites :
 - %ROWTYPE (schéma d'une table ou résultat de requête),
 - RECORD (type complexe dérivé),
 - TABLE (tables dynamiques)
 - Large Object :
 - LOB (jusqu'à 4 Go)
- Variables non PL/SQL : définies sous SQL*Plus (de substitution et globales), variables hôtes (déclarées dans des programmes précompilés).

Nouveaux types PL/SQL

- Nouveaux types prédéfinis :
 - **BINARY_INTEGER** : entiers signés entre -2^{31} et 2^{31}
 - **PLS_INTEGER** : entiers signés entre -2^{31} et 2^{31} ; plus performant que **NUMBER** et **BINARY_INTEGER** au niveau des opérations arithmétiques ; en cas de dépassement, exception levée
- Sous-types PL/SQL : restriction d'un type de base
 - Prédéfinis : **CHARACTER**, **INTEGER**, **NATURAL**, **POSITIVE**, **FLOAT**, **SMALLINT**, **SIGNTYPE**, etc.
 - Utilisateur (restriction : précision ou taille maximale) :
SUBTYPE nomSousType IS typeBase [(contrainte)] [NOT NULL] ;
- Exemple de sous-type utilisateur :
SUBTYPE numInsee IS NUMBER(13) NOT NULL;

Déclaration des variables

```
identifieur [CONSTANT] datatype [NOT NULL]
           [:= | DEFAULT expr];
```

- **Identificateur :**

- Variable, curseur, exception, etc.
- Commence par une lettre
- Peut contenir : lettres, chiffres, \$, #, _
- Interdits : &, -, /, *espace*
- Jusqu'à 30 caractères
- Insensible à la casse

(nometudiant = NomETUDIANT)

- **Commentaires :**

-- **Commentaire sur une seule ligne**

/* **Commentaire sur plusieurs lignes** */

Variables simples

- Variables de type SQL

```
nbr      NUMBER(2) ;  
nom      VARCHAR(30) ;  
minimum  CONSTANT INTEGER := 5 ;  
salaire  NUMBER(8,2) ;  
debut    NUMBER NOT NULL ;
```

- Variables de type booléen (TRUE, FALSE, NULL)

```
fin      BOOLEAN ;  
reponse  BOOLEAN DEFAULT TRUE ;  
ok       BOOLEAN := TRUE;
```

Exemples de variables scalaires

- Number, Boolean, Date, Time, Varchar2, etc.

```
DECLARE
  v_job          VARCHAR2(9) ;
  v_count        BINARY_INTEGER := 0;
  v_total_sal    NUMBER(9,2) := 0;
  v_orderdate    DATE := SYSDATE + 7;
  c_tax_rate     CONSTANT NUMBER(3,2) := 8.25;
  v_valid        BOOLEAN NOT NULL := TRUE;
  ...
```

- Affectation:
 - Variable := Expression
 - Par le directive **DEFAULT**
 - Par la directive INTO d'une requête (SELECT ... **INTO** *variable* FROM ...).

Les variables scalaires

- Déclaration de variables %type
 - Déclarer une variable selon la définition d'une colonne d'une table ou d'une vue existante ou une autre variable précédemment déclarée.

```
identifier      Table.column_name%TYPE;
```

```
...  
  v_name          employees.last_name%TYPE;  
  v_balance       NUMBER(7,2);  
  v_min_balance   v_balance%TYPE := 10;  
...
```

Conversions

- Conversions implicites :
 - Lors du calcul d'une expression ou d'une affectation
 - Si la conversion n'est pas autorisée, une exception est levée
- Conversions explicites :

De	A	CHAR	NUMBER	DATE
CHAR			TO_NUMBER	TO_DATE
NUMBER		TO_CHAR		TO_DATE
DATE		TO_CHAR		

Les variables composites: Record

- Syntaxe 1

```
TYPE type_name IS RECORD
    (field_declaration[, field_declaration]...);
identifier    type_name;
```

```
field_name {field_type | variable%TYPE
            | table.column%TYPE | table%ROWTYPE}
            [[NOT NULL] {:= | DEFAULT} expr]
```

- Exemple 1

```
...
TYPE emp_record_type IS RECORD
    (last_name    VARCHAR2(25),
     job_id       VARCHAR2(10),
     salary       NUMBER(8,2));
emp_record       emp_record_type;
...
```

Les variables composites: Record

- **Syntaxe 2 :**

Soit par référence à une structure de table en utilisant la notion %ROWTYPE

Nom_varibale nom_table%rowtype;

- **Exemple 2:**

```
dept_record    departments%ROWTYPE;
```

Variables TABLE

- Définir des tableaux dynamiques (dimension initiale non précisée) composés d'une clé primaire et d'une colonne de type scalaire, **%TYPE**, **%ROWTYPE** ou **RECORD**.

```
TYPE    nomTypeTableau    IS    TABLE    OF    {typeScalaire    |  
variable%TYPE    |    table.colonne%TYPE    |    table%ROWTYPE    |  
nomTypeRecord} [NOT NULL] [INDEX BY BINARY_INTEGER];
```

```
nomVariableTableau nomTypeTableau;
```

- Si **INDEX BY BINARY_INTEGER** est présente, l'index peut être entre -2^{31} et 2^{31} (type **BINARY_INTEGER**)

Variables TABLE

- Déclaration d'un type tableau

```
TYPE <nom du type du tableau>  
IS TABLE OF <type de l'élément>  
INDEX BY BINARY_INTEGER;
```

- Affectation (héritage) de ce type à une variable

```
<nom élément> <nom du type du tableau>;
```

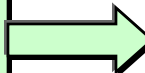
- Utilisation dans la section BEGIN : un élément du tableau :

```
<nom élément> (rang dans le tableau)
```

Tableaux dynamiques variables simples

- Déclaration d'un tableau avec des éléments numériques

```
TYPE  type_note_tab  
IS TABLE OF NUMBER(4,2)  
INDEX BY BINARY_INTEGER;  
tab_notes type_note_tab;  
i NUMBER;
```



```
i:=1;  
tab_notes(i) := 12.5;
```

- Déclaration d'un tableau avec des éléments caractères

```
TYPE  type_nom_tab  
IS TABLE OF VARCHAR(30)  
INDEX BY BINARY_INTEGER;  
tab_noms type_nom_tab;  
i NUMBER;
```

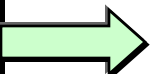


```
i:=1;  
tab_noms(i) := 'toto';
```

Tableaux dynamiques variables simples avec héritage

- Tableau avec éléments hérités

```
TYPE type_note_tab  
IS TABLE OF partiel.note%TYPE  
INDEX BY BINARY_INTEGER;  
tab_notes type_note_tab;  
i NUMBER;
```



```
i:=1;  
tab_notes(i) := 12.5;
```

```
TYPE type_nom_tab  
IS TABLE OF etudiant.nom%TYPE  
INDEX BY BINARY_INTEGER;  
tab_noms type_nom_tab;  
i NUMBER;
```

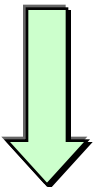


```
i:=1;  
tab_noms(i) := 'toto';
```

Tableaux dynamiques avec des éléments de type RECORD

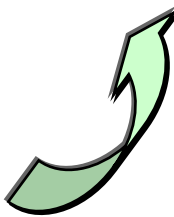
- Type RECORD : plusieurs variables dans un élément

```
TYPE  type_emp_record  
IS RECORD  
(idEmp NUMBER,  
 nomEmp VARCHAR(30),  
 adrEmp VARCHAR(80));
```



```
TYPE  type_emp_tab  
IS TABLE OF type_emp_record  
INDEX BY BINARY_INTEGER;  
tab_ems type_emp_tab;  
i NUMBER;
```

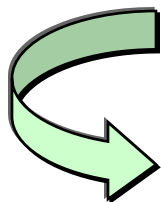
```
i:=1;  
tab_ems(i).idEmp:= 100;  
tab_ems(i).nomEmp:= 'toto';  
tab_ems(i).adrEmp:= 'tlse';
```



Tableaux dynamiques avec des éléments de type ROW

- Type ROW : chaque élément est une variable 'struct'

```
TYPE  type_emp_tab  
IS TABLE OF employee%ROWTYPE  
INDEX BY BINARY_INTEGER;  
tab_emps type_emp_tab;  
i NUMBER;
```



```
i:=1;  
tab_emps(i).idE:= 100;  
tab_emps(i).nom:= 'toto';  
tab_emps(i).adresse:= 'tlse';
```

Fonctions (Tableaux)

- Fonctions PL/SQL dédiées aux tableaux : **EXISTS(x)**, **PRIOR(x)**, **NEXT(x)**, **DELETE(x,...)**, **COUNT**, **FIRST**, **LAST**, **DELETE**

nom_table_sql.COUNT	retourne le nombre d'éléments de la table
nom_table_sql.DELETE	supprime toute la table
nom_table_sql.DELETE (n)	supprime l'élément n
nom_table_sql.DELETE (n,m)	supprime les éléments de n à m
nom_table_sql.EXISTS (n)	retourne TRUE si l'élément n existe
nom_table_sql.FIRST	retourne le premier index
nom_table_sql.LAST	retourne la valeur du dernier index
nom_table_sql.NEXT(n)	élément suivant
nom_table_sql.PRIOR(n)	élément précédent

Portée des variables

- Un bloc peut être imbriqué dans le code d'un autre bloc (on parle de sous-bloc). Un sous-bloc peut aussi se trouver dans la partie des exceptions. Un sous-bloc commence par BEGIN et se termine par END.
- La portée d'un objet (variable, exception, etc.) est la zone du programme qui peut y accéder. Un bloc qui déclare qu'un objet peut y accéder, ainsi que les sous-blocs. En revanche, un objet déclaré dans un sous-bloc n'est pas visible du bloc supérieur (principe des accolades de C et Java).

```
...  
  x  BINARY_INTEGER;  
BEGIN  
  ...  
  DECLARE  
    y  NUMBER;  
  BEGIN  
    y := x;  
  END;  
  ...  
END;
```

Scope of x

Scope of y

Portée des variables

```
<<outer>>
DECLARE
  V_SAL          NUMBER(7,2) := 60000;
  V_COMM         NUMBER(7,2) := V_SAL * .20;
  V_MESSAGE      VARCHAR2(255) := ' eligible for commission';
BEGIN
  DECLARE
    V_SAL          NUMBER(7,2) := 50000;
    V_COMM         NUMBER(7,2) := 0;
    V_TOTAL_COMP   NUMBER(7,2) := V_SAL + V_COMM;
  BEGIN
    V_MESSAGE := 'CLERK not' || V_MESSAGE;
    outer.V_COMM := V_SAL *.30
  END;
  V_MESSAGE := 'SALESMAN' || V_MESSAGE;
END;
```

① →

② →

- La valeur de V_message, V_Comm à la position 1 et 2

Variables de substitution

- Variables lues par un ACCEPT PROMPT

+

PL

```
ACCEPT plu PROMPT 'Entrer la valeur : '  
  
DECLARE  
    -- déclarations  
BEGIN  
    -- travail avec le contenu de plu :  
    -- &plu si numérique  
    -- '&plu' si caractère  
END;  
/
```

+

Exemple

```
SQL> ACCEPT s_matPilote PROMPT 'Matricule : '  
SQL> ACCEPT s_nomPilote PROMPT 'Nom pilote : '  
SQL> ACCEPT s_agePilote PROMPT 'Age pilote : '  
DECLARE  
    salairePilote NUMBER(6,2) DEFAULT 32000;  
BEGIN  
    INSERT INTO pilote VALUES ('&s_matPilote',  
                                '&s_nomPilote', NULL, &s_agePilote,  
                                salairePilote);  
END;  
/
```

Variables hôtes, variables de session

- Variables hôtes :
 - Définies en dehors de PL/SQL et appelées dans PL/SQL
 - Dans le code PL/SQL, préfixer le nom de la variable de « : »

Instructions PL

- Affectation (:=)
 - A := B;
- Structure alternative ou conditionnelle
 - Opérateurs SQL : >, <,, OR, AND,, BETWEEN, LIKE, IN
 - IF THEN ELSEEND IF;

```
IF condition THEN
    instructions;
ELSE
    instructions;
    IF condition THEN instructions;
        ELSIF condition THEN instructions;
            ELSE instructions;
END IF;
```

Traitements itératifs : la boucle de base

- Ensemble d'instructions écrites une seule fois et exécutées à plusieurs reprises.
- PL/SQL permet d'effectuer des traitements répétitifs grâce à la clause **LOOP**.
- Quatre types de boucles:
 - La boucle de base
 - La boucle **FOR**
 - La boucle **WHILE**
 - La boucle **CURSOR ... FOR**

Traitements conditionnels

- Exécution d'instructions en fonction du résultat d'une condition

```
IF condition1 THEN traitement1;  
  {ELSIIF condition2 THEN traitement2;}  
  [ELSE traitement3;]  
END IF ;
```

● Règles

- Dès que l'une des conditions est vraie, exécution du traitement qui suit le **THEN**
- Si aucune condition n'est vraie, exécution du traitement **ELSE**
- Seules les clauses **IF**, **THEN**, **END IF** sont obligatoires

Structure alternative : CASE

- Choix selon la valeur d'une variable

```
CASE variable  
  
    WHEN valeur1 THEN action1;  
    WHEN valeur2 THEN action2;  
    .....  
    ELSE    action;  
  
END CASE;
```

Structure itérative

- LOOP

```
LOOP
    instructions;
    EXIT WHEN (condition);
END LOOP;
```

- FOR

```
FOR (indice IN [REVERSE] borne1..borne2) LOOP
    instructions;
END LOOP;
```

- WHILE

```
WHILE (condition) LOOP
    instructions;
END LOOP;
```

L'ordre Select dans PL/SQL

```
SELECT  select_list
INTO    {variable_name[, variable_name]...
        | record_name}
FROM    table
[WHERE  condition];
```

```
DECLARE
    v_deptno          NUMBER(4);
    v_location_id     NUMBER(4);
BEGIN
    SELECT      department_id, location_id
    INTO        v_deptno, v_location_id
    FROM        departments
    WHERE       department_name = 'Sales';
    ...
END;
/
```

La boucle FOR: Exemple

● Exemple: Calcul de la Factorielle 9

```
DECLARE  
fact NUMBER := 1;  
BEGIN  
  FOR i IN 1..9  
  LOOP  
    Fact:= Fact*i;  
  END LOOP ;  
  INSERT INTO Resultat  
  VALUES (Fact, 'factorielle9');  
END;
```

La boucle While: Exemple

- Exemple: Calcul du reste de la division 7324 par 9

```
DECLARE
```

```
Reste NUMBER := 7324;
```

```
BEGIN
```

```
  WHILE reste >= 9
```

```
  LOOP
```

```
    reste := reste - 9;
```

```
  END LOOP ;
```

```
  INSERT INTO Resultat
```

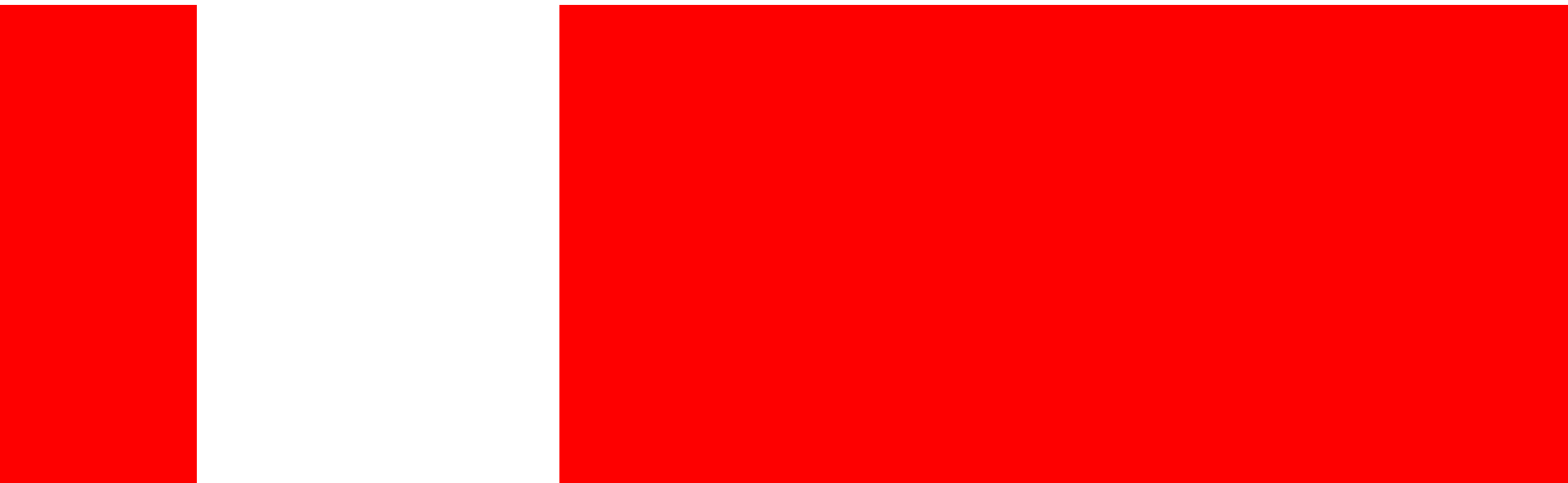
```
  VALUES (reste, 'reste division de 7324 par 9');
```

```
END ;
```

Package DBMS_OUTPUT

- Le package DBMS_OUTPUT permet d'afficher des messages à partir des blocs PL/SQL. Il assure la gestion des entrées/sorties de blocs ou sous-programmes PL/SQL.
 - PUT: permet d'ajouter des informations dans la ligne en cours.
 - NEW_LINE: écriture du caractère fin de ligne.
 - PUT_LINE: PUT + NEW_LINE.

```
DBMS_OUTPUT.PUT_LINE('Affichage des n premiers ');  
DBMS_OUTPUT.PUT_LINE('caractères en ligne ');  
FOR i IN 1..n LOOP  
    DBMS_OUTPUT.PUT(tab_cars(i));  
END LOOP;  
DBMS_OUTPUT.NEW_LINE;
```

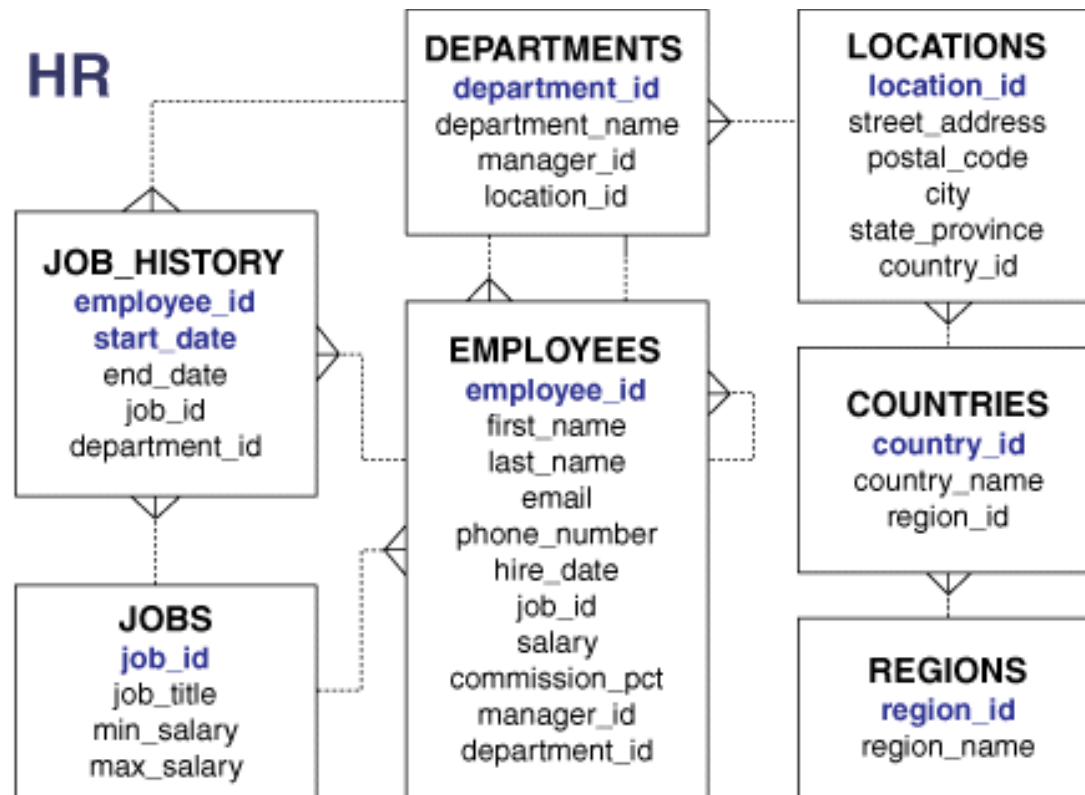


ORACLE®

Questions ???

Exercices

- Soit la base de données suivante:



Exercices

- Écrire votre premier bloc PLSQL (affichant bonjour).
- Écrire un bloc PLSQL qui affiche le salaire de l'employé 100.
- Modifier votre bloc pour saisir le numéro de l'employé.

Exercices

- Écrire un bloc PLSQL qui affiche le tableau de multiplication de 8 et 9.

- -- table de multiplication par 8 et par 9...

```
declare
    type tablemul is record ( par8 number, par9
    number);
    type tabledentiers is table of tablemul index by
    binary_integer;
    ti tabledentiers;
    i number;
begin
    for i in 1..10 loop
        ti(i).par9 := i*9 ;
        ti(i).par8:= i*8;
        dbms_output.put_line (i||'*8='||ti(i).par8||'
' || i || '*9=' || ti(i).par9 );
    end loop;
end;
/
```

Pour plus de détails

TYPE	VALEURS
BINARY-INTEGER	entiers allant de -2^{31} à 2^{31})
POSITIVE / NATURAL	entiers positifs allant jusqu'à $2^{31} - 1$
NUMBER	Numérique (entre -2^{418} à 2^{418})
INTEGER	Entier stocké en binaire (entre -2^{126} à 2^{126})
CHAR (n)	Chaîne fixe de 1 à 32767 caractères (différent pour une colonne de table)
VARCHAR2 (n)	Chaîne variable (1 à 32767 caractères)
LONG	idem VARCHAR2 (maximum 2 gigaoctets)
DATE	Date (ex. 01/01/1996 ou 01-01-1996 ou 01-JAN-96 ...)
RAW	Permet de stocker des types de données binaire relativement faibles(≤ 32767 octets) idem VARCHAR2. Les données RAW ne subissent jamais de conversion de caractères lors de leur transfert entre le programme et la base de données.
LONG RAW	Idem LONG mais avec du binaire

Recommendations

Objet	Convention	Exemple
Variable	v _nomVariable	v_compteur
Constante	c _nomConstante	c_pi
Exception	e _nomException	e_pasTrouvé
Type RECORD	nomRecord_ rec	pilote_rec
Variable RECORD	r _nomVariable	r_pilote
Variable ROWTYPE	rty _nomVariable	rty_pilote
Type-tableau	nomTypeTableau_ tytab	pilotes_tytab
Variable tableau	tab _nomTableau	tab_pilotes
Curseur	nomCurseur_ cur	pilotes_cur
Variable de substitution (SQL*Plus)	s _nomVariable	s_brevet
Variable de session (globale)	g _nomVariable	g_brevet